

Fiche du projet

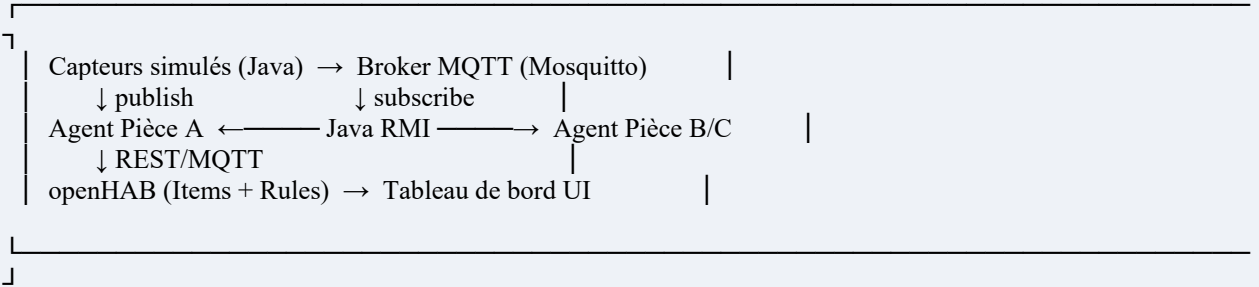
Durée : 3 semaines (travail en équipe de 2 ou 3 étudiants)
Technologies : Java 21, Eclipse Mosquitto (MQTT), openHAB 4, Java RMI
Concepts couverts: IoT, Middleware, Systèmes multi-agents, Communication IPC/RMI
Rendu final : Code source (GitHub/ZIP) + rapport PDF + démonstration (10 min)
Date limite : Fin de la semaine 3 (selon calendrier affiché sur Moodle)

1. Contexte et mise en situation

Les bâtiments intelligents reposent sur des systèmes distribués capables de collecter des données de capteurs hétérogènes, de les analyser en temps réel et d'automatiser des actions sur des actuateurs physiques. Dans ce projet, vous concevez et implémentez une plateforme SmartAgent-IoT qui simule un étage de bâtiment intelligent composé de plusieurs pièces, chacune dotée de capteurs (température, luminosité, occupation) et d'actuateurs (thermostat, éclairage, volets).

Chaque pièce est gérée par un agent logiciel autonome. Les agents communiquent entre eux via Java RMI (appels de méthodes distants entre JVM) et échangent des mesures de capteurs via MQTT (protocole publish/subscribe léger). openHAB sert de couche d'intégration et de tableau de bord central pour visualiser l'état du bâtiment et déclencher des règles d'automatisation.

Architecture cible



- Broker MQTT : Eclipse Mosquitto (port 1883)
- Topics MQTT : `batiment/{piece}/capteur/{type}` et `batiment/{piece}/actuateur/{type}`
- Java RMI : port 2199 — chaque agent expose une interface `IAgentPiece`
- openHAB : Things MQTT + Items + DSL Rules

2. Planification — Trois semaines de travail

S1	Infrastructure IoT & MQTT	Semaines 1
----	---------------------------	------------

Objectifs semaine 1

- Installer et configurer Eclipse Mosquitto (broker MQTT local)
- Implémenter des simulateurs de capteurs Java publiant sur MQTT
- Implémenter des abonnés Java lisant les mesures et les affichant
- Configurer openHAB avec Things MQTT, Items et une règle DSL simple

Tâche 1.1 — Simulateur de capteurs MQTT

Précisions — simulateurs MQTT

Vous devez simuler exactement trois pièces : salleA, salleB et cuisine.

Chaque pièce publie trois mesures toutes les 5 secondes : température, luminosité et occupation.

Les valeurs peuvent être aléatoires, mais elles doivent rester dans des plages réalistes que vous documenterez.

Validation : utilisez un client MQTT ou les logs pour vérifier séparément les neuf flux de données (3 pièces × 3 types de mesure).

Créez un projet Maven sensor-simulator. Pour chaque pièce (SalleA, SalleB, Cuisine), implémentez une classe CapteurSimulateur qui publie toutes les 5 secondes sur les topics suivants :

Topics MQTT — Convention de nommage

batiment/salleA/capteur/temperature	→ ex: 22.4
batiment/salleA/capteur/luminosite	→ ex: 340
batiment/salleA/capteur/occupation	→ ex: true
batiment/salleB/capteur/temperature	→ ex: 19.8
...	

Squelette — CapteurSimulateur.java

```
// Extrait - classe à compléter
public class CapteurSimulateur implements Runnable {
    private final String piece;
    private final MqttClient client;
    private final Random rnd = new Random();

    @Override
    public void run() {
        while (true) {
            publishTemp();
            publishLuminosite();
            publishOccupation();
            Thread.sleep(5000);
        }
    }
    // TODO : implémenter publishTemp(), publishLuminosite(), publishOccupation()
}
```

Tâche 1.2 — Configuration openHAB

Précisions — openHAB

Créez les fichiers batiment.things, batiment.items et batiment.rules dans les dossiers de configuration openHAB appropriés.

Configurez au minimum les trois capteurs pour chaque pièce et un canal de commande thermostat.

Au moins deux règles DSL doivent être fonctionnelles dans la version finale : une règle d'économie d'énergie et une règle d'alerte.

Validation semaine 1 : les valeurs MQTT doivent être visibles dans openHAB et au moins une règle doit pouvoir être déclenchée de manière reproductible.

Configurez openHAB 4 pour recevoir les mesures MQTT. Créez les fichiers de configuration dans `conf/things/` et `conf/items/` :

batiment.things (extrait)

```
// batiment.things
Thing mqtt:broker:mosquitto [host="localhost", port=1883]
Thing mqtt:topic:salleA (mqtt:broker:mosquitto) {
    Channels:
        Type number : temperature [stateTopic="batiment/salleA/capteur/temperature"]
        Type number : luminosite  [stateTopic="batiment/salleA/capteur/luminosite"]
        Type switch  : occupation  [stateTopic="batiment/salleA/capteur/occupation",
                                   on="true", off="false"]
}
```

batiment.rules (exemple DSL)

```
// batiment.rules - règle DSL : baisser chauffage si pièce vide
rule "Economie energie salleA"
when
    Item SalleA_Occupation changed to OFF
then
    SalleA_Thermostat.sendCommand(18) // consigne 18°C quand vide
end
```

S2	Agents distribués Java RMI	Semaine 2
-----------	-----------------------------------	-----------

Objectifs semaine 2

- Concevoir l'interface distante `IAgentPiece` avec Java RMI
- Implémenter un `AgentPiece` (serveur RMI) par pièce, souscrit au MQTT
- Permettre aux agents de se consulter mutuellement via RMI (partage d'état)
- Implémenter un coordinateur central (`AgentCoordinateur`) via RMI

Tâche 2.1 — Interface distante `IAgentPiece`

Contrat RMI à respecter

Utilisez les mêmes signatures de méthodes dans l'interface et dans toutes les implémentations. La méthode de résumé d'état est nommée `getEtatJson()`.

Toutes les méthodes distantes doivent déclarer `RemoteException`.

Le registre RMI utilise le port 2199 et chaque agent doit être enregistré sous un nom distinct, par exemple `agent-salleA`.

IAgentPiece.java — interface RMI

```
// Interface distante - à implémenter
public interface IAgentPiece extends Remote {
    double getTemperature() throws RemoteException;
    int    getLuminosite()  throws RemoteException;
}
```

```

    boolean isOccupee()           throws RemoteException;
    void    setConsigneTemperature(double t) throws RemoteException;
    String  getEtat()             throws RemoteException; // JSON résumé
}

```

Tâche 2.2 — AgentPiece (serveur RMI + abonné MQTT)

Responsabilité d'un AgentPiece

Un AgentPiece représente une seule pièce. Il s'abonne uniquement aux topics de cette pièce et conserve la dernière valeur reçue pour chaque capteur.

setConsigneTemperature(double t) doit publier la nouvelle consigne sur bâtiment/{pièce}/actuateur/thermostat.

Les champs partagés entre callback MQTT et appels RMI doivent être gérés de manière thread-safe.

Validation : depuis un client RMI, lire les trois états d'une pièce puis modifier sa consigne de thermostat.

AgentPiece.java — squelette

```

public class AgentPiece extends UnicastRemoteObject implements IAgentPiece {
    private volatile double temperature;
    private volatile int    luminosite;
    private volatile boolean occupation;
    private final String nom;

    // Abonnement MQTT (dans le constructeur)
    // client.subscribe("batiment/" + nom + "/capteur/#", (topic, msg) -> {
    //     ... mettre à jour les champs volatiles ...
    // });

    // TODO : implémenter tous les getters + setConsigneTemperature
    // setConsigneTemperature publie sur batiment/{nom}/actuateur/thermostat
}

```

Tâche 2.3 — Coordinateur central

Politique d'équilibrage à implémenter

Considérez au minimum les paires adjacentes salleA–salleB et salleB–cuisine.

Si l'écart absolu de température dépasse 4 °C, le coordinateur calcule une consigne cible et envoie une commande à l'agent concerné.

Journalisez la paire, les deux températures, la décision et la consigne envoyée.

Validation semaine 2 : créer volontairement un écart supérieur à 4 °C et montrer que la commande est envoyée via RMI puis publiée sur MQTT.

Le CoordinateurBatiment se connecte à tous les agents via RMI et applique une politique d'équilibrage thermique : si la différence entre deux pièces adjacentes dépasse 4°C, il demande un ajustement de consigne.

CoordinateurBatiment.java — squelette

```

public class CoordinateurBatiment {
    private final Map<String, IAgentPiece> agents = new HashMap<>();
}

```

```
public void equilibrerTemperatures() throws RemoteException {
    // Pour chaque paire de pièces adjacentes :
    // si abs(tA - tB) > 4.0 → ajuster la consigne de la plus chaude
    // TODO
}
```

S3	Intégration, tests & rapport	Semaine 3
----	------------------------------	-----------

Objectifs semaine 3

- Intégrer le coordinateur RMI avec les règles openHAB (via REST API openHAB)
- Écrire des tests d'intégration (simuler scénarios réels)
- Tester la tolérance aux pannes : déconnecter un agent MQTT et observer
- Rédiger le rapport final et préparer la démonstration

Tâche 3.1 — Test de tolérance aux pannes

Définition du timeout et comportement attendu

Considérez un capteur en timeout lorsqu'aucun nouveau message n'a été reçu pendant 30 secondes.

La panne d'une pièce ne doit pas arrêter les autres agents ni le coordinateur.

Le coordinateur doit gérer RemoteException et continuer son cycle avec les agents encore disponibles.

Lors de la démonstration, montrez clairement l'état avant la panne, la détection, puis le retour à un fonctionnement normal.

Coupez le client MQTT d'une pièce (simulez une panne capteur). Vérifiez que :

- Les autres agents continuent de fonctionner normalement
- Le coordinateur détecte le timeout et passe à la dernière valeur connue
- openHAB affiche un état UNDEF sur l'Item correspondant

Tâche 3.2 — Scénario de démonstration

Script de démonstration (10 minutes)

1. Lancer Mosquitto + openHAB + 3 agents RMI (AgentPiece) [2 min]
2. Montrer les mesures en temps réel dans l'UI openHAB [1 min]
3. Simuler une pièce vide → règle DSL baisse la consigne automatiquement [2 min]
4. Créer un déséquilibre de température → coordinateur équilibre [2 min]
5. Couper un agent MQTT → timeout + récupération [2 min]
6. Questions de l'enseignante [1 min]

3. Livrables attendus

Livrable	Description	Pondération
L1 — Code source	Projet Maven multi-module (sensor-simulator, agent-piece, coordinateur) + conf openHAB	40 %
L2 — Rapport PDF	Architecture, choix de conception,	30 %

	difficultés rencontrées, captures d'écran (8-12 pages)	
L3 — Démonstration	Présentation live de 10 min devant l'enseignante avec questions-réponses	20 %
L4 — Qualité du code	Javadoc, gestion d'exceptions, tests JUnit, lisibilité	10 %

4. Barème détaillé — /100 points

Critère	Description	Points
MQTT — Pub/Sub	Capteurs publient correctement, agents s'abonnent et lisent les valeurs	15 pts
openHAB Things	Things MQTT + Items correctement configurés, mesures visibles dans UI	10 pts
openHAB Rules	≥ 2 règles DSL fonctionnelles (économie énergie + alerte)	10 pts
Java RMI	Interface distante + 3 serveurs AgentPiece enregistrés dans le registre RMI	20 pts
Coordinateur	Politique d'équilibrage thermique correcte via RMI	10 pts
Tolérance pannes	Gestion timeout MQTT + reprise sur erreur RemoteException	10 pts
Rapport PDF	Architecture claire, diagrammes, analyse critique des choix	15 pts
Démonstration	Fluidité, réponses aux questions, maîtrise du système	10 pts

⚠ Tout plagiat ou code généré sans compréhension et sans commentaires explicatifs entraîne une note de 0 pour l'ensemble du projet.